

Calibrate

Job-Skill Gap Analyzer for Turkish CS Students
Project Introduction & Requirements Report (Revised)

1 Executive Summary

Calibrate is a web application that analyzes the gap between a university student’s current skill profile and real-time job market demand in Turkey. Students upload their CV, select target roles and receive a ranked, explainable breakdown of missing skills, market trends and a personalized learning roadmap, all grounded in live data from Turkish job postings and the ESCO multilingual skill ontology.

The system is built in two stages with a clear boundary between them.

1. A deterministic data stage scrapes Turkish job postings, aggregates skill demand per role cluster, embeds both the market demand and the student’s CV skills, and detects gaps by semantic similarity. Every demand figure here is counted, not estimated.
2. An agentic stage consumes that grounded output and turns it into ranked, explainable recommendations and a learning roadmap. The agent reasons and narrates. It does not produce the underlying numbers.

The core of the product is the combination of live Turkish market data, embedding-based skill matching that handles Turkish and English wording, and an explainable recommendation agent hosted on AWS. Existing tools such as LinkedIn Premium are English-first, generic and blind to local market dynamics, whereas Calibrate is built for the Turkish job market from the ground up.

2 Problem Statement

2.1 The Gap Nobody Is Measuring

Turkish computer science and engineering graduates face a paradox. Employers report difficulty hiring qualified candidates while graduates struggle to find relevant positions.

- The disconnect centers on invisible, unmeasured skill mismatches.
- Students frequently optimize for academic grades rather than market-relevant skills.
- Career advisors lack real-time data on active company hiring requirements.
- Job postings use inconsistent skill terminology, which makes self-assessment unreliable. The same skill appears as “Veri Bilimi”, “Data Science” and “veri bilimci”, and keyword matching treats these as unrelated.
- No existing tool maps a specific student’s profile directly to specific Turkish employer needs.

The terminology problem above is the direct reason the matching stage uses embeddings rather than exact keyword or string matching.

2.2 Why Existing Solutions Fall Short

Tool	What It Does	Critical Weakness	Turkish Market?
LinkedIn Premium	Skill gap for job listings	English-first, generic, no trend data	No
Kariyer.net	Job listings only	No skill analysis or CV matching	Yes
Ada / Coursera Career	Course recommendations	Not grounded in a specific job market	No
University Career Offices	Manual advising	No data, no scale, no real-time signal	Partial
Calibrate (proposed)	Grounded gap analysis + trends	None identified	Built for it

3 Product Vision & Goals

3.1 Vision Statement

Every Turkish CS student should be able to upload their CV on Monday and know exactly what to learn by Tuesday, based on what companies in their city are hiring for right now, not six months ago.

3.2 Primary Goals

- Detect the exact skills missing between a student's profile and their target job market segment, using semantic similarity rather than keyword overlap.
- Surface which skills are trending upward in Turkish job postings over time.
- Deliver explainable, actionable output instead of black-box scores.
- Support both Turkish and English CV input and job posting analysis.
- Let students search live postings by entering qualifications, reusing the same embedding index (stretch goal).
- Be demonstrable as a working product within 8 weeks.

4 System Architecture

4.1 High-Level Components

Layer	Component	Technology
Data	Job Posting Scraper	Python (Playwright + BeautifulSoup), Kariyer.net
Data	Skill Ontology	ESCO v1.2 (EU Open Data, multilingual, TR + EN)
Data	Demand Aggregator	Per-role-cluster skill frequency and trend deltas, stored as artifacts in S3
ML	CV Text Extraction	PDF/DOCX to text, skill phrases via spaCy + BERTurk
ML	Embedding Model	Multilingual embeddings (Amazon Bedrock, or a BERTurk-based sentence-transformer)
ML	Gap Detector	Cosine similarity between CV skill vectors and aggregated demand vectors
ML	Posting Index	Per-posting embeddings with vector search (FAISS or pgvector) for qualification search
ML	Trend Detector	Skill frequency over weekly crawl windows
Agent	Recommendation Agent	Strands Agents on Amazon Bedrock AgentCore Runtime
Backend	API Server	FastAPI (Python), REST endpoints
Frontend	Web Application	React + Recharts for visualization
Infra	Cloud	AWS (Bedrock, AgentCore Runtime, S3)

5 Functional Requirements

• FR-01: CV Upload & Parsing

- System accepts CVs in PDF and DOCX formats, in Turkish and English.
- The pipeline extracts name, education, GPA, work experience, projects, listed skills and certifications.
- Extracted skills are normalized against the ESCO taxonomy and embedded into vectors for matching.
- The parser handles Turkish characters without data loss.

• FR-02: Job Market Data Collection

- The scraper collects Turkish CS and engineering postings from Kariyer.net on a scheduled basis, using Playwright to render pages and BeautifulSoup to parse them.
- Each posting stores title, company, location, required skills, preferred skills and date posted.
- Skill terms are normalized to ESCO concept URIs and aggregated into a per-role-cluster demand profile.

- Minimum dataset target is 5,000 postings pre-collected before the demo, with a live weekly refresh.
- **FR-03: Skill Gap Analysis**
 - The system embeds the student’s skills and the aggregated demand for their target role cluster, then computes the gap by semantic similarity. A demanded skill with no close match in the CV is a gap.
 - Gap output is ranked by market frequency, surfacing the most demanded missing skills first.
 - Each gap item includes the skill name, demand percentage, ESCO category, the closest CV skill considered (or its absence) and suggested learning resources.
 - Output is human-readable and explainable, with no raw numeric scores shown.
- **FR-04: Trend Detection**
 - System tracks skill frequency over weekly snapshot crawl windows.
 - Skills rising more than 15% month-over-month are flagged as “Emerging”.
 - Skills in decline are flagged as “Fading” so students can deprioritize them.
 - Trend data is visualized as a time-series chart in the frontend.
- **FR-05: Role-Based Segmentation**
 - Users select one or more target roles: Backend Engineer, Frontend Engineer, Data Scientist, ML Engineer, DevOps, Full Stack or Product Engineer.
 - Role clusters define how postings are bucketed during aggregation, so gap analysis is scoped to the selected cluster rather than the whole market.
 - Multi-role selection produces a union of required skills with role-specific frequency weighting.
- **FR-06: Multilingual Support**
 - A multilingual embedding model matches Turkish and English skill wording directly, so synonyms across languages map to nearby vectors.
 - The user interface is available in Turkish and English.
 - ESCO mappings use both TR and EN concept labels for cross-language consistency.
- **FR-07: Learning Roadmap Generation**
 - For each gap skill, the agent suggests one to three free or open resources (Coursera, edX, YouTube, official documentation), drawn only from a resource lookup tool.
 - Resources are filtered for Turkish availability where possible.
 - Roadmaps are downloadable as a PDF summary.
- **FR-08: Recommendation Agent**

- A Strands agent hosted on Amazon Bedrock AgentCore Runtime consumes the gap list, the aggregated role demand and the trend labels, then produces ranked, explainable recommendations and the roadmap.
 - The agent reasons only over data returned by its tools, `get_role_demand` and `get_learning_resources`. It never invents demand figures or resources.
 - Tools are plain Python functions exposed to the agent. The system prompt requires every recommendation to state its market-frequency reason and forbids showing raw scores.
- **FR-09: Qualification-Based Job Search (stretch goal)**
 - A user enters a set of qualifications or skills, in Turkish or English, as free text or selected items.
 - The system embeds the input and returns live postings ranked by semantic similarity, with the matched skills highlighted and each posting’s required and preferred skills shown.
 - This reuses the embedding pipeline but requires per-posting embeddings stored in a vector index (FAISS or pgvector), since it returns individual postings rather than the aggregate.
 - The same search can optionally be exposed as an agent tool, so recommendations can cite real example postings.
 - This feature is gated behind the core CV flow and is built only if the core lands on schedule.

6 Data Sources

Source	Purpose	Access Method
Kariyer.net	Turkish job postings	Scrape with Playwright + BeautifulSoup, pre-cached
ESCO v1.2	Multilingual ontology	EU Open Data download
BERTurk	Turkish NLP and embeddings	HuggingFace download
Sentence-Transformers	Semantic embeddings	HuggingFace / pip install
Amazon Bedrock	Embedding and foundation models	Managed AWS service
Stack Overflow Survey	Trend baseline	Public annual CSV download
Synthetic CVs	Model evaluation only	Team-generated

7 Eight-Week Delivery Plan

Week	Phase	Deliverables	Owner
1	Data Foundation	Scraper live (Playwright + BS4), ESCO indexed, 5K postings collected, demand aggregator v1	SWE + ML
2	Embedding Pipeline	CV text and skill extraction, ESCO normalization, embedding of CV and demand skills	ML
3	Gap Engine	Similarity-based gap detection, ranked gap output, CLI demo	ML + SWE
4	Trend + Eval	Time-series trend labels, evaluation (extraction F1, match accuracy, Spearman)	ML
5	Recommendation Agent	Strands agent on AgentCore, tools and system prompt, grounded recommendations	ML + SWE
6	Frontend + Deploy	React app (upload, role selector, dashboard, trend charts), agent on AgentCore Runtime	SWE
7	Integration	End-to-end testing, Turkish localization, performance tuning, observability	Full Team
8	Demo & Pitch	Demo video, pitch deck, GitHub docs, rehearsals	Full Team
S	Stretch	Qualification-based posting search (FR-09), built only if the core flow lands on time	ML + SWE

8 Team Structure & Roles

Role	Primary Responsibilities	Key Outputs
ML Engineer(s)	Embedding pipeline, gap detector, trend model, recommendation agent	Embedder, similarity matcher, trend labels, Strands agent, evaluation report
SWE, Backend	Scraping, aggregation, FastAPI, AgentCore deployment	Scraper, demand artifacts, REST API, agent runtime
SWE, Frontend	React app, visualization, UX	Upload UI, gap dashboard, trend charts, localization
All	Documentation, testing, demo prep	README, pitch deck, demo script, final presentation